

Ledgerly — Product Requirements Document

v1.0 · July 2026 · Owner: Arjun · Status: Approved for build

Companion docs: *Ledgerly Architecture & Database Design* · Interactive prototype (*Ledgerly.dc.html*) is the visual + behavioral spec

1 • Problem & vision

Existing tools each solve half the problem: Splitwise handles group splits but has no real personal finance; Money Manager/Walnut/Monito track personal spend but can't settle with flatmates. Ledgerly is one system: **a personal ledger first, a shared-expense platform second**, on the same data.

Vision statement: every rupee I earn, hold, spend, or am owed — visible in one place, in under 3 taps to log, with my flatmates able to join later without me migrating anything.

2 • Users & context

- **P0 — Owner (me):** salaried, Bengaluru, 5 payment instruments (2 banks, cash, UPI wallet, credit card), currently on Monito. Logs 5-10 transactions/day. Wants dashboards that answer "what needs my attention today".
- **P1 — Flatmates (Phase 2):** 2-3 friends. Join via invite. Care only about shared expenses, balances, and settling. Must be able to participate *before* registering (ghost participants).

Currency & locale: INR only at launch, en-IN digit grouping (₹1,23,456) everywhere. Timezone Asia/Kolkata.

3 • Goals & non-goals

Goals

- Log an expense in ≤ 3 taps / ≤ 5 seconds
- Account-level truth: balances always reconcile
- Decision-oriented dashboard (bills due, budgets breached, who owes whom)
- Full Monito history imported losslessly
- Shared expenses added without redesign (schema ready day 1)
- Works well on phone; installable PWA

Non-goals (explicit)

- **No AI/LLM/ML anywhere** — all features deterministic and rule-based (standing product rule)
- No bank account linking / statement sync at launch
- No multi-currency at launch
- No native mobile apps (responsive web + PWA)
- No monetization features
- No OCR at launch (receipt images stored as-is)

4 • Functional requirements — Phase 1 (Personal)

Priority: P0 = launch blocker, P1 = fast-follow. Each requirement has acceptance criteria (AC).

4.1 Accounts & transfers — P0

- CRUD financial accounts: type (bank / cash / UPI wallet / credit card / investment), bank name, nickname, opening balance, color/icon, archive.
- Internal transfers between any two accounts, including credit-card bill payment; transfers are excluded from income/expense totals.
- **AC:** after any sequence of operations, $\text{balance} = \text{openingBalance} + \sum \text{ledger}$ holds for every account; deleting a transaction restores balances exactly.

4.2 Transactions — P0

- Expense fields: amount, account, category, subcategory, merchant, date+time, payment method, tags, notes, receipt, location (optional), recurring flag, personal/shared.
- Income sources: salary, bonus, freelancing, cashback, interest, refund, gift, rental, investment returns, other.
- Quick-add: FAB and ☰K palette from any screen; amount + category + account is enough (merchant defaults from category rule table). Soft delete with 5-second undo toast.
- **AC:** add-expense happy path ≤ 3 interactions; every transaction row shows split/recurring/receipt badges as in the prototype.

4.3 Categories — P0

- 18 seeded defaults (Food, Groceries, Rent, Electricity, Water, Gas, Internet, Fuel, Shopping, Medical, Travel, Entertainment, Education, Insurance, Investment, EMI, Subscription, Misc) + custom categories and one level of subcategories.
- Rule-based auto-categorization: merchant→category mapping table, seeded with a common-merchant dictionary, updated on every manual re-categorization. **AC:** re-categorizing a merchant twice makes the third occurrence auto-assign.

4.4 Budgets — P0

- Weekly / monthly / yearly, per category or per account or overall. Progress bars with 80% (amber) and 100% (red) thresholds.
- **AC:** crossing a threshold creates a notification exactly once per budget per period; breached budgets appear in the dashboard attention strip.

4.5 Bills & recurring — P0

- Bills with due date, recurrence, reminder offsets (7d / 3d / due day); "mark paid" creates the payment transaction and rolls the due date.
- Recurring transactions: daily / weekly / monthly / quarterly / yearly with interval; materialized by the daily job, idempotent.
- **AC:** a bill due within 7 days appears in the attention strip with correct urgency color; a recurring rule never double-creates for a period.

4.6 Dashboard — P0

- As built in the prototype: attention strip; Total Balance (dark card) + Income + Expenses + Saved/rate; cash flow (income vs expense, 6M/8W/14D toggle); accounts widget; category donut; upcoming bills; settlements; recent transactions; budget bars. "Net Worth" label is reserved until liabilities tracking exists.
- **AC:** dashboard answers "what needs my action today" above the fold on a 390px viewport.

4.7 Search — P0

- Filter by merchant, category, amount range, notes, account, tags, payment method, date range, type. ☞K palette with the deterministic phrase parser ("swiggy in march", "upi expenses", "income in june") compiling to the same filters.
- **AC:** parser answers with total + count inline and one click opens the filtered list; unparseable queries fall back to full-text search — never an error state.

4.8 Analytics & reports — P1

- Monthly spending trend, category breakdown, merchant analysis, avg daily spend, balance trend, budget usage, payment-account usage, expense heatmap (P1).
- Reports for day/week/month/quarter/year/custom range; export CSV, XLSX, PDF.

4.9 Receipts — P1

- Upload (camera or file), view, replace, delete; private storage with signed URLs; 10 MB / image-or-PDF allowlist.

5 • Functional requirements — Phase 2 (Shared)

- **Participants & groups — P0:** add friends by name (no account required); groups (e.g. "Flat 402") with owner/admin/member roles; email invitations that link a ghost participant to a real user on acceptance, preserving history.
- **Splits — P0:** equal, percentage, exact, ratio, custom; exclude participants; payer can be anyone. Stored truth is exact paise per participant summing to the total; remainder paise go to the payer. **AC:** ₹2,500 ÷ 4 equal = 625 each; ₹1,000 ÷ 3 equal = 334/333/333 with the 334 on the payer.
- **Settlement engine — P0:** net balances per person; minimum-transaction suggestion (greedy max-debtor→max-creditor); record settlements (UPI/cash/bank) with history; partial settlements allowed. **AC:** after settling all suggested transactions every net balance is zero.
- **Notifications — P1:** budget exceeded, bill due, settlement reminder, recurring created, monthly summary, large-expense alert (threshold configurable). In-app first; email digest P2.

6 • Functional requirements — Phase 3 (Import)

- **Monito import — P0 for migration:** CSV/XLSX wizard: upload → structure auto-detection (header heuristics + value-shape scoring) → column mapping UI → category mapping (remembered in ImportMapping) → preview with duplicate detection (date+amount+merchant hash, ±1 day) → row-level validate/skip/edit → commit as a batch.
- **Import history & undo:** every batch records counts + errors; one-click undo soft-deletes the whole batch. **AC:** import → undo → re-import produces zero duplicates.
- **Generic engine:** adapter interface so Splitwise / Walnut / bank-CSV adapters are additive modules.

7 • UX requirements

- Visual spec = prototype: "Glass & Ink" — porcelain white, #2a63f6 accent, Schibsted Grotesk, dark mode toggle (persisted), 14px radius cards.
- Responsive: sidebar ≥ 760px; bottom tab bar + FAB + bottom-sheet modals below. Touch targets ≥ 44px.
- ⌘K command palette (all actions + search), keyboard shortcuts, toast notifications with undo, skeleton loaders, designed empty states for every list, WCAG 2.1 AA (contrast, focus rings, semantic landmarks, full keyboard operability).

8 • Non-functional requirements

Area	Requirement
Performance	Dashboard LCP < 2s on 4G; transaction list paginated (cursor, 50/page); optimistic mutations.
Correctness	All money integer paise; split sums enforced by DB constraint; balances rebuildable; unit tests on settlement + split rounding + parser.
Security	Better Auth sessions; per-user scoping in services + Supabase RLS; zod validation; rate limiting on auth/import; signed URLs for receipts; audit log on money-bearing writes.
Privacy	No third-party analytics on financial data; full data export (CSV/JSON) and account deletion self-serve.
Reliability	Daily Supabase backups; recurring job idempotent; import undo-able.

9 • Success metrics & release criteria

- **Adoption (self):** 100% of daily spends logged for 30 consecutive days; Monito fully retired after M6.

- **Speed:** median add-expense time < 5s (instrumented client-side).
- **Trust:** zero balance-reconciliation discrepancies over the first 90 days.
- **Phase 2 gate:** ship shared features only after 30 days of stable personal use.
- **Release criteria per milestone (M1-M6, see Architecture doc §7):** ACs above pass, unit tests green on money math, no P0 bugs open.

10 • Open questions

- Credit-card statement cycles: model as simple negative-balance account (current plan) or add statement periods + due tracking in Phase 1.5?
- PWA offline logging (queue writes, sync later) — M4 or later?
- Should ghost participants see a read-only share link of their balance before signing up?